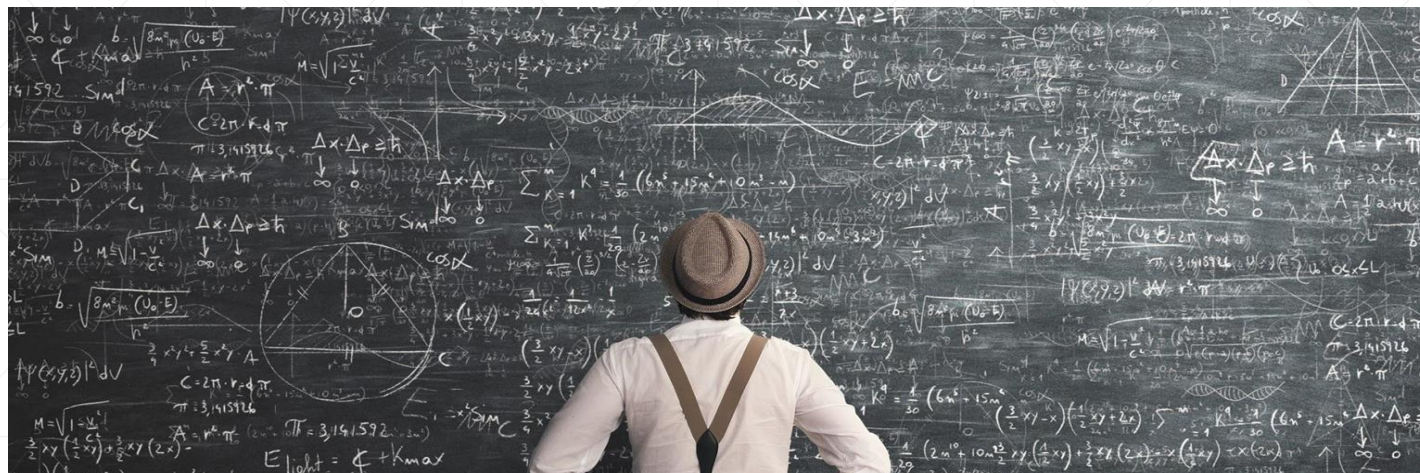


多窗体编程之二



窗体之间的信息交换

北京理工大学计算机学院
金旭亮

概述



实际开发中，应用程序所创建的两个窗体对象中，需要交换信息的方式不外乎以下几种：



信息的单向传送：

A将相应信息传给B窗体，然后显示B窗体

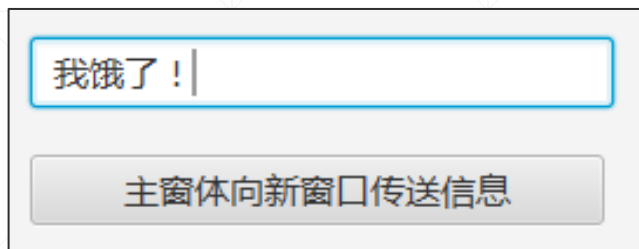
A打开B窗体，用户在B窗体输入一些信息，然后点击“确定”或“取消”按钮关闭B窗体，A窗体得到相应的信息。



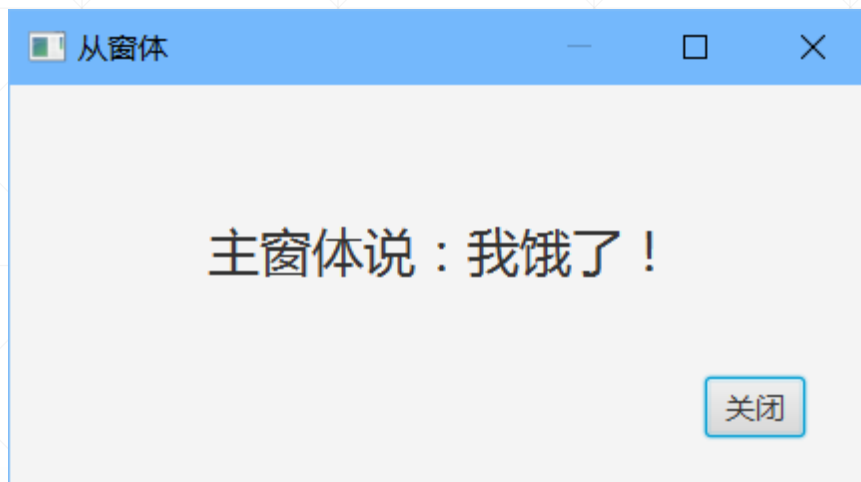
信息的双向传送：

屏幕上A和B窗体同时显示，用户操作A窗体，B窗体响应，反过来也是这样。

信息交换实例：主窗体向从窗体传送信息



主窗体



从窗体要接收数据，必须在其控制器中有相应的数据接收方法。

实例分析

从窗体控制器:接收信息

```
public class OtherWindowController {  
  
    private Stage stage;  
  
    public void init(Main main, Stage stage) {  
        this.stage=stage;  
    }  
    //关闭从窗体自己  
    public void close() {  
        stage.close();  
    }  
    @FXML Label infoLabel;  
    //从窗体用于从外部接收信息的方法  
    public void setMessage(String message) {  
        infoLabel.setText(message);  
    }  
}
```

负责接收信息的从窗体控制器，要提供一个方法接收信息（通常是利用方法的参数）

主窗体控制器:发送信息

```
//主窗体向另一个窗体发送信息  
public void mainToOther() {  
  
    String message="主窗体说: ";  
    if(userInput.getText().length()>0){  
        message += userInput.getText();  
    }  
    else {  
        message+="用户没有输入";  
    }  
    main.mainToOther(message);  
}
```

main变量引用Main类的实例。
具体完成主窗体向从窗体发送信息工作，
由Main类中的mainToOther()方法实现的，
实现代码见下页

主类中实现主窗体向从窗体消息的传送

```
// 主窗体向从窗体传送信息
public void mainToOther(String message) {

    try {
        FXMLLoader loader = new FXMLLoader(
            Main.class.getResource("OtherWindowView.fxml"));
        AnchorPane root = (AnchorPane) loader.load();
        Stage secondStage = new Stage();
        // 启用模态对话框
        secondStage.initModality(Modality.APPLICATION_MODAL);
        OtherWindowController controller = loader.getController();
        controller.init(this, secondStage);
        //发送信息
        controller.setMessage(message);
        //显示从窗体
        Scene scene = new Scene(root);
        secondStage.setScene(scene);
        secondStage.setTitle("从窗体");
        secondStage.show();
    } catch (IOException e) {

        e.printStackTrace();
    }

}
```

通常我们在Main类中放置切换显示各窗体，并在各窗体所关联的控制器中相互交换信息的相应代码。

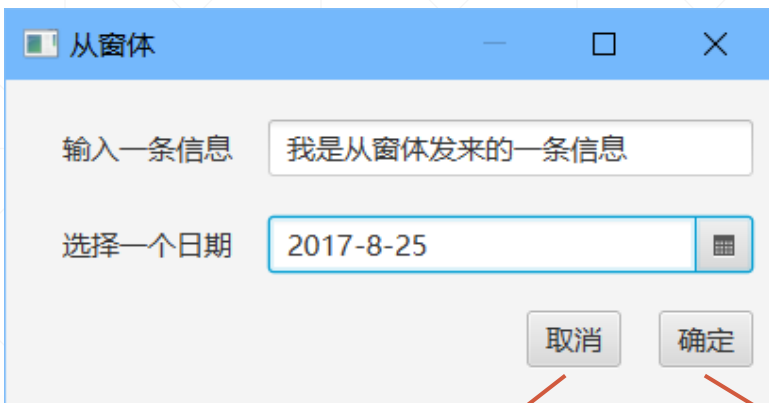
消息的传送，是通过获取接收消息的窗体所关联的控制器对象引用，调用其所提供的“接收消息”方法而实现的。

信息交换实例：从窗体将信息传回给主窗体

要点：

1. 主窗体new一个从窗体显示并接收用户输入。
2. 主窗体需要区分两种状态（Ok和Cancel），以便做出相应的处理。

要实现这种信息交换方式，关键在于主窗体控制器的引用要传给从窗体控制器。



主窗体从新窗口中接收信息

用户取消了输入

主窗体

主窗体从新窗口中接收信息

2017-08-25:我是从窗体发来的一条信息

主窗体

主窗体控制器

```
//显示另一个窗体
public void showThirdWindow() {
    main.thirdWindow(this);
}

@FXML Label messageLabel;
//从另一个窗体中提取信息
public void showMessage(String message) {
    messageLabel.setText(message);
}
```

主窗体控制器中，要定义一个公有方法，本例中为showMessage()，外界可以调用它将消息传给主窗体控制器。



此例中的showMessage()，主窗体控制器并不会“主动调用”它，而是由外界（比如其它窗体控制器）所“**回调 (call back)**”，这种编程方法非常典型，注意掌握。

从窗体控制器中的代码

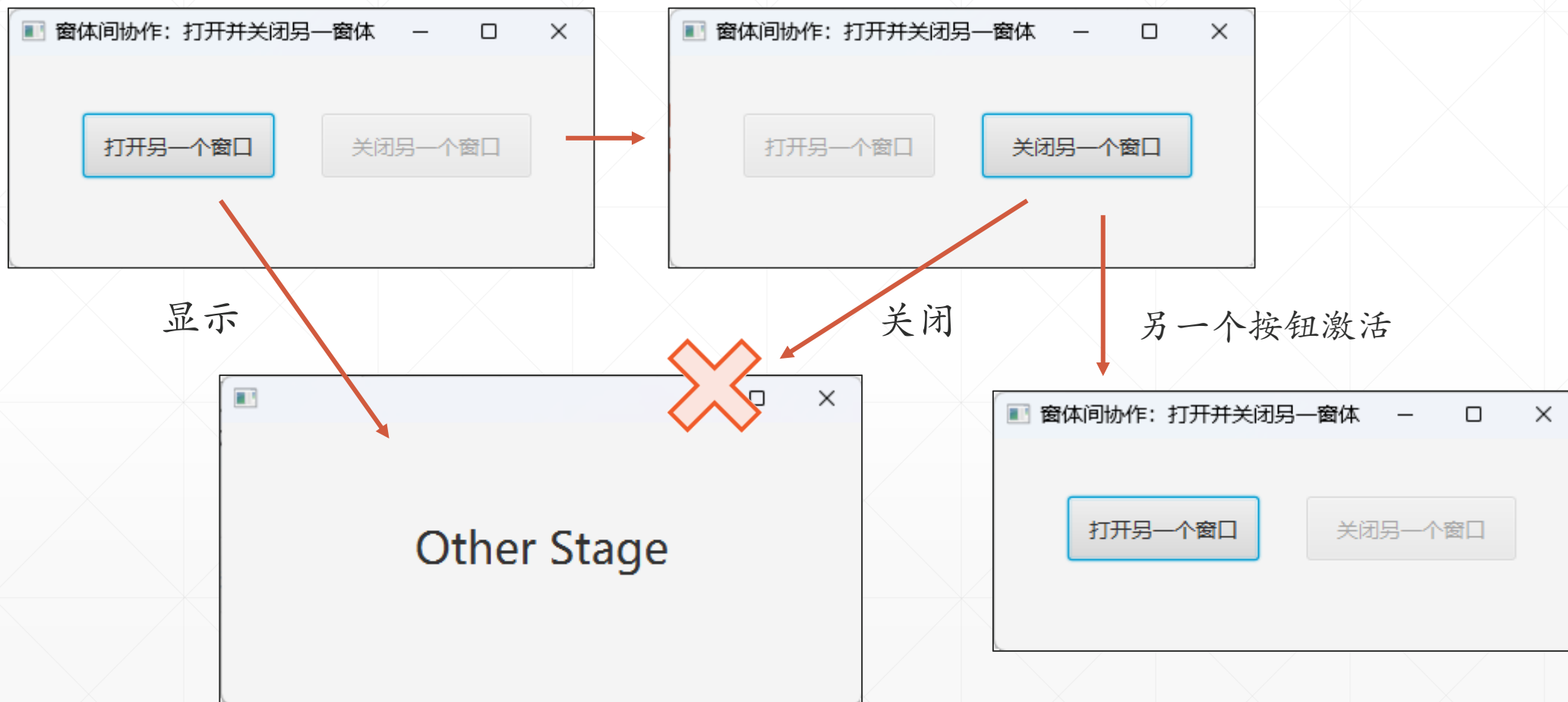
```
public class ThirdWindowController {  
  
    @FXML DatePicker datePicker;  
    @FXML TextField messageTextField;  
  
    private Stage myStage;  
    //引用主窗体控制器  
    private MainWindowController mainWindowController;  
  
    public void init(MainWindowController controller, Stage stage) {  
        this.mainWindowController=controller;  
        myStage=stage;  
    }  
  
    private String result;  
    //生成要传送的消息  
    public String getResult() {  
        result=datePicker.getValue()+":"+messageTextField.getText();  
        return result;  
    }  
}
```

注意一下，init()方法接收外界的主窗体控制器对象引用。

```
public void cancel() {  
    result="用户取消了输入";  
    //直接调用主窗体控制器的相应方法传送信息  
    mainWindowController.showMessage(result);  
    myStage.close();  
}  
  
public void ok() {  
    result=getResult();  
    //直接调用主窗体控制器的相应方法传送信息  
    mainWindowController.showMessage(result);  
    myStage.close();  
}
```

ok与cancel两个方法，直接“回调”主窗体控制器的“信息接收方法”，从而实现信息在两个对象之间的传送。

示例：如何关闭另一窗体？



示例：CloseOtherStage

基本思路



要关闭一个窗体，其实就是调用Stage的close()方法，此方法，又会去调用其基类Window的hide()方法。



问题的关键，在于控制器中的代码，如何获取Stage对象的引用？



两种方法：

(1) 在装载fxml时，获取控制器的引用，然后，将Stage对象的引用注入进去

(2) 利用控制器中标有@FXML的控件引用获取Stage对象的引用：

```
var stage=(Stage)控件变量.getScene().getWindows();
```

示例分析-1

```
private void openOtherStage() {  
    var loader = new FXMLLoader(getClass().getResource("other.fxml"));  
    try {  
        AnchorPane root = loader.load();  
        //保存控制器引用  
        otherController = loader.getController();  
        Scene scene = new Scene(root);  
        var stage = new Stage();  
        stage.setScene(scene);  
        stage.show();  
        //禁用相应的按钮  
        btnOpenNewStage.setDisable(true);  
        btnCloseStage.setDisable(false);  
    } catch (IOException ex) {  
        throw new RuntimeException(ex);  
    }  
}
```

示例分析-2

```
private void closeOtherStage() {  
    if(otherController!=null){  
        otherController.closeStage();  
        btnCloseStage.setDisable(true);  
    }  
    btnOpenNewStage.setDisable(false);  
}
```

MainController.java

```
public class OtherController {  
  
    2 usages  
    @FXML  
    private Label lblInfo;  
  
    1 usage  
    public void closeStage(){  
        lblInfo.getScene().getWindow().hide();  
    }  
  
}
```

小结



- ✓ 本讲通过两个示例展示了JavaFX中实现多窗体编程的基本技巧，主要目的是起一个启发思路的作用。
- ✓ 事实上，实现主从窗体之间的信息交换，是有多种实现方式的，本讲只是介绍了典型的实现方案。
- ✓ 另外，本讲只是介绍了“一对一”的单向信息交换，在实际开发中，还会遇到“一对一双向实时”、“一对多”、“多对一”和“多对多”信息交换这些复杂的开发场景。
- ✓ 如何实现上述功能，留给大家作为课后练习。